

ISTVT: Interpretable Spatial-Temporal Video Transformer for Deepfake Detection

EE656 Course Project Report — Reproduction & Adaptation

Authors

Ojas Bajpai	240712
Pratyush Juyal	240787
Priyanshu Yadav	240812
Snehansh Maharaj	241025
Gourav Singh Chouhan	240408

Course: Deep Learning (EE656)
Instructor: Prof. Nishchal K. Verma
Institution: Indian Institute of Technology Kanpur

Abstract

This report documents our course project implementation of **ISTVT (Interpretable Spatial-Temporal Video Transformer)** for deepfake detection, proposed by Zhao et al. in IEEE Transactions on Information Forensics and Security (2023). The project reproduces the paper’s core architectural contributions—the **decomposed spatial-temporal self-attention** and the **self-subtract mechanism**—in a Google Colab notebook targeting the free-tier T4 GPU. The implementation uses the **Celeb-DF-v2** dataset (590 real / 590 fake videos, balanced). Because the paper’s from-scratch Xception backbone requires far more training data than is available, the feature extractor is replaced with a **pretrained EfficientNet-B0** backbone, while the transformer architecture, classification token design, and LRP-based Algorithm 1 visualization are all implemented faithfully from the paper. Training ran for 20 epochs on 944 training videos (6,608 clips), achieving a **clip-level accuracy of 93.46%** and a **video-level AUC of 0.9959** on the held-out validation set. LRP heatmaps were generated and confirm that spatial attention localises blending boundaries while temporal attention tracks inter-frame flickering in moving facial regions.

Contents

1	Introduction and Project Scope	3
2	Paper Background: ISTVT Architecture	3
2.1	Overview	3
2.2	Decomposed Spatial-Temporal Self-Attention	3
2.3	Self-Subtract Mechanism	4
2.4	Interpretability: LRP Visualization (Algorithm 1)	4
3	Our Implementation	4
3.1	Environment and Dataset	4
3.2	Faithful vs. Adapted Components	5
3.3	Backbone Substitution: Pretrained EfficientNet-B0	5
3.4	Decomposed Spatial-Temporal Attention: Code-Level Details	6
3.5	Full ISTVT Model	7
3.6	Data Preprocessing Pipeline	7
3.6.1	Face Detector: OpenCV DNN instead of MTCNN	8
3.6.2	Preprocessing Steps	8
3.7	Dataset Design: Multiple Overlapping Clips	8
3.8	Training Configuration	8
3.9	Evaluation Protocol	9
4	LRP Visualization: Implementation of Algorithm 1	9
4.1	Overview	9
4.2	Implementation Steps	9
4.3	Visualization Output	9
5	Key Design Decisions and Challenges	9
5.1	Consecutive Frame Sampling	9
5.2	Video-level Train/Val Split	10
5.3	Mixed Precision (AMP)	10
5.4	Challenges Summary	10
6	Model Configuration Summary	10
7	Results and Analysis	10

7.1	Quantitative Results	10
7.2	Comparison with the Original Paper	11
7.3	Analysis of Results	11
8	LRP Heatmap Visualization Results	12
9	Conclusion	13

1 Introduction and Project Scope

Deepfake videos, generated by per-frame face-manipulation algorithms such as DeepFakes, FaceSwap, Face2Face, and NeuralTextures, pose a growing threat to digital media authenticity. Frame-based detectors that learn spatial artifacts in isolation are limited: they overfit to the specific synthesis method they were trained on, degrade on compressed video, and cannot exploit the temporal inconsistencies that are a fundamental signature of per-frame synthesis.

Project Goal

Reproduce the core contribution of Zhao et al. (2023)—the **decomposed spatial-temporal self-attention** and **self-subtract mechanism** of ISTVT—as a complete, runnable Google Colab notebook, adapting where necessary to operate on a single T4 GPU with the Celeb-DF-v2 dataset instead of the paper’s 4-GPU FaceForensics++ setup.

The paper’s two key claims motivate the reproduction:

1. Decomposing spatial and temporal self-attention reduces complexity from $\mathcal{O}(T^2H^2W^2)$ to $\mathcal{O}(T^2 + H^2W^2)$ while matching or exceeding vanilla video transformer performance.
2. The self-subtract mechanism—which forces temporal attention to attend to inter-frame *residuals* rather than raw features—is the single most impactful component in the paper’s ablation study.

Our implementation scope is limited to ISTVT itself. The paper’s baselines (VTN, VidTr, ViViT, STIL, FTCN) are **not** re-implemented per the assignment instructions.

2 Paper Background: ISTVT Architecture

2.1 Overview

ISTVT is a three-stage pipeline: (1) a compact CNN feature extractor, (2) M spatial-temporal transformer blocks with decomposed attention, and (3) an MLP prediction head on a joint classification token.

Given a frame sequence $\mathbf{S} \in \mathbb{R}^{T \times C_i \times H_i \times W_i}$, the CNN produces feature maps $\mathbf{F} \in \mathbb{R}^{T \times C \times H \times W}$, flattened to tokens $\mathbf{T} \in \mathbb{R}^{T \times HW \times C}$. Two classification tokens are appended:

- **Spatial CLS** $\mathbf{T}_s \in \mathbb{R}^{T \times 1 \times C}$: one per frame.
- **Temporal CLS** $\mathbf{T}_t \in \mathbb{R}^{1 \times (HW+1) \times C}$: one row.

After concatenation and learnable positional embedding, the tensor $\mathbf{I} \in \mathbb{R}^{(T+1) \times (HW+1) \times C}$ flows through M transformer blocks. The joint CLS token at $(0, 0, :)$ is fed to the MLP head for binary prediction.

2.2 Decomposed Spatial-Temporal Self-Attention

Why Decompose?

Deepfake synthesis processes frames *independently*, so spatial artifacts are largely independent across frames and should be modelled per-frame. Temporal attention should capture only the *cross-frame* signal. Decomposing the two avoids the joint cost $\mathcal{O}(T^2H^2W^2)$ and enables separate spatial/temporal interpretability.

For **temporal** self-attention (rearranged to $N \times (HW+1) \times (T+1) \times D$):

$$\mathbf{O}_{(:,j,:,:) }^t = \text{softmax} \left(\frac{\mathbf{Q}_{(:,j,:,:) } \mathbf{K}_{(:,j,:,:) }^\top}{\sqrt{D}} \right) \mathbf{V}_{(:,j,:,:) } \quad (1)$$

For **spatial** self-attention (rearranged to $N \times (T+1) \times (HW+1) \times D$):

$$\mathbf{O}_{(k,::,:)}^s = \text{softmax}\left(\frac{\mathbf{Q}_{(k,::,:)}\mathbf{K}_{(k,::,:)}^\top}{\sqrt{D}}\right)\mathbf{V}_{(k,::,:)} \quad (2)$$

Complexity reduces from $\mathcal{O}(T^2H^2W^2)$ to $\mathcal{O}(T^2 + H^2W^2)$.

2.3 Self-Subtract Mechanism

Motivation

Static regions (background, stable facial areas) dominate temporal attention unless suppressed. The self-subtract operation replaces raw tokens with frame-to-frame *residuals*, forcing temporal attention to ask “what changed?”—exactly the signal that reveals per-frame synthesis inconsistencies.

For $\mathbf{I} \in \mathbb{R}^{(T+1) \times (HW+1) \times C}$:

$$\mathbf{I}' = \text{cat}\left(\left[\begin{array}{c} \mathbf{I}[0:2, :, :] \\ \mathbf{I}[2 :, :, :] - \mathbf{I}[1: -1, :, :] \end{array}\right], \text{dim} = 0\right) \quad (3)$$

Boundary Condition at $t = 0$: Formal Treatment

The token sequence at dimension 0 is $[\mathbf{T}_t; \mathbf{F}_1; \mathbf{F}_2; \dots; \mathbf{F}_T]$, where $\mathbf{T}_t$ is at index 0 and frame tokens $\mathbf{F}_t$ are at indices $1, \dots, T$. The operation is precisely:

$$\begin{aligned} \mathbf{I}'[0, :, :] &= \mathbf{I}[0, :, :] \quad (\text{temporal CLS preserved}) \\ \mathbf{I}'[1, :, :] &= \mathbf{I}[1, :, :] \quad (\text{frame 1 token preserved}) \\ \mathbf{I}'[t, :, :] &= \mathbf{I}[t, :, :] - \mathbf{I}[t-1, :, :], \quad t = 2, 3, \dots, T \end{aligned} \quad (4)$$

Preserving $\mathbf{I}[1]$ avoids a spurious strong response at $t = 1$ that would arise from differencing against a zero-padded pseudo-frame. $\mathbf{I}'$ is used only for Q and K projections; V is projected from the original $\mathbf{I}$, retaining full spatial content.

2.4 Interpretability: LRP Visualization (Algorithm 1)

Per-block relevance:

$$\bar{A}_{d(i,::,:)}^{(m)} = \mathbf{I} + \max\left(\mathbb{E}_h\left[R_{d(:,i,::,:)}^{(m)} \circ \nabla A_{d(:,i,::,:)}^{(m)}\right], 0\right) \quad (5)$$

Cumulative relevance across all M blocks:

$$U_d^{(i,::,:)} = \bar{A}_{d(i,::,:)}^{(1)} \cdot \bar{A}_{d(i,::,:)}^{(2)} \cdots \bar{A}_{d(i,::,:)}^{(M)} \quad (6)$$

U_t and U_s are reshaped and bilinearly upsampled to the original resolution, giving separate spatial and temporal heatmaps.

3 Our Implementation

3.1 Environment and Dataset

Celeb-DF-v2 was downloaded from Kaggle directly into the Colab VM. With 590 real and 5,639 fake videos available, we enforce a strict 50/50 balance (590 real + 590 fake) to prevent class imbalance from masking model behaviour. The dataset splits into 944 training videos and 236 validation videos.

Why Celeb-DF-v2 instead of FaceForensics++

The paper evaluates on FaceForensics++ (FF++). We use Celeb-DF-v2 because it is freely accessible on Kaggle, whereas the full FF++ dataset requires a signed research agreement.

Table 1: Environment and Dataset Configuration

Resource / Choice	Details
GPU	NVIDIA Tesla T4 (15 GB VRAM), Google Colab free tier
Framework	PyTorch 2.11 + <code>timm</code> , <code>einops</code> , <code>tqdm</code>
Dataset	Celeb-DF-v2 (balanced: 590 real + 590 fake videos)
Face detector	OpenCV DNN (SSD + ResNet-10)
Sequence length T	6 consecutive frames (same as paper)
Frame resolution	128×128 (paper: 300×300)
Frames cached per video	32 (paper: up to 270)
Total params	5,154,813 (of which 5,153,885 trainable)

Our implementation is dataset-agnostic: only the Kaggle dataset ID and folder names differ.

3.2 Faithful vs. Adapted Components

Table 2: Faithful vs. Adapted Architecture Components

Component	Paper	Our Implementation & Reason
Decomposed spatial-temporal attention	Temporal-first (Eqs. 1–2)	Identical. Temporal first, then spatial.
Self-subtract mechanism	Before temporal Q/K (Eq. 3)	Identical. Q, K from $\mathbf{I}'$; V from $\mathbf{I}$.
Classification tokens	Spatial CLS + temporal CLS	Identical. Same dual-token design.
MLP prediction head	LayerNorm $\rightarrow$ Linear on CLS	Identical.
LRP visualization (Alg. 1)	Separate spatial/temporal maps	Implemented exactly.
Transformer depth M	12 blocks	6 blocks. 12 at embed=128 causes T4 OOM.
Embed dimension C	728 (Xception)	128. Proportional to smaller extractor.
Feature map size	$19 \times 19 = 361$ tokens	$8 \times 8 = 64$ tokens. Reduces attention cost.
Input resolution	300×300	128×128 . VRAM & disk constraint.
Feature extractor	Xception (from scratch, FF++)	Pretrained EfficientNet-B0. Too little data for from-scratch CNN (Sec. 3.3).
Frames per video	Up to 270 (FF++)	32 frames cached. Colab disk limit.
Optimizer	SGD, single LR 5×10^{-4}	AdamW, layer-wise LR. Pretrained backbone needs separate LR (Sec. 3.8).
Dataset	FaceForensics++ (4K videos)	Celeb-DF-v2 (590+590). Availability.

3.3 Backbone Substitution: Pretrained EfficientNet-B0

Why not Xception from scratch?

The paper trains Xception from random initialization on $\sim 4,000$ FF++ videos. Our balanced Celeb-DF-v2 split gives only **944 training videos**. Training from scratch would not converge to meaningful texture representations. We substitute with **pretrained EfficientNet-B0** (`timm`, ImageNet weights), fine-tuned at a low learning rate. The transformer—the paper’s actual contribution—is unaffected.

The PretrainedTextureExtractor module: loads EfficientNet-B0 with `features_only=True`, `out_indices=(2,)`, freezes the first 2 stages, applies a 1×1 Conv + BN + ReLU projection to 128 channels, then adaptive average pooling to an 8×8 grid.

Listing 1: PretrainedTextureExtractor: backbone loading and projection

```

1 class PretrainedTextureExtractor(nn.Module):
2     def __init__(self, embed_dim=128, backbone_name="tf_efficientnet_b0",
3                 feature_map_hw=8, freeze_backbone_stages=2):
4         super().__init__()
5         self.backbone = timm.create_model(
6             backbone_name, pretrained=True,
7             features_only=True, out_indices=(2,))
8         backbone_out_channels = self.backbone.feature_info.channels()[-1]
9         for i, child in enumerate(self.backbone.children()):
10            if i < freeze_backbone_stages:
11                for p in child.parameters():
12                    p.requires_grad = False
13        self.proj = nn.Conv2d(backbone_out_channels, embed_dim, kernel_size=1)
14        self.bn = nn.BatchNorm2d(embed_dim)
15        self.act = nn.ReLU(inplace=True)
16        def forward(self, x):                                # x: (B*T, 3, H, W)
17            feat = self.backbone(x)[-1]
18            feat = self.act(self.bn(self.proj(feat)))
19            feat = F.adaptive_avg_pool2d(feat, (self.feature_map_hw)*2)
20            return feat                                       # (B*T, embed_dim, 8, 8)

```

3.4 Decomposed Spatial-Temporal Attention: Code-Level Details

Listing 2: Self-subtract and decomposed attention (forward pass)

```

1 def forward(self, x):
2     B, Tp1, HWp1, C = x.shape
3     # Self-subtract (Eq. 3): preserve indices 0:2, diff the rest
4     x_sub = torch.cat([
5         x[:, 0:2, :, :],                                # temporal CLS + frame 1
6         x[:, 2:, :, :] - x[:, 1:-1, :, :]                # frame residuals for t >= 2
7     ], dim=1)
8     # Temporal attention: Q, K from residuals; V from original
9     q_t = rearrange(self.q_temp(x_sub), "b t s (h d) -> b h s t d", h=self.
10                    num_heads)
11     k_t = rearrange(self.k_temp(x_sub), "b t s (h d) -> b h s t d", h=self.
12                    num_heads)
13     v_t = rearrange(self.v_temp(x), "b t s (h d) -> b h s t d", h=self.
14                    num_heads)
15     attn_t = (q_t @ k_t.transpose(-2, -1)) * self.scale
16     attn_t = attn_t.softmax(dim=-1)
17     self.attn_t_weights = attn_t                          # retained for LRP
18     if attn_t.requires_grad:
19         attn_t.retain_grad()
20     out_t = rearrange(self.attn_drop(attn_t) @ v_t, "b h s t d -> b t s (h d)"
21                      )
22
23     # Spatial attention: all from temporal output
24     q_s = rearrange(self.q_spatial(out_t), "b t s (h d) -> b h t s d", h=self.
25                    num_heads)
26     k_s = rearrange(self.k_spatial(out_t), "b t s (h d) -> b h t s d", h=self.
27                    num_heads)
28     v_s = rearrange(self.v_spatial(out_t), "b t s (h d) -> b h t s d", h=self.
29                    num_heads)
30     attn_s = (q_s @ k_s.transpose(-2, -1)) * self.scale
31     attn_s = attn_s.softmax(dim=-1)
32     self.attn_s_weights = attn_s
33     if attn_s.requires_grad:

```

```

27     attn_s.retain_grad()
28     out_s = rearrange(self.attn_drop(attn_s) @ v_s, "b h t s d -> b t s (h d)"
29 )
29     return self.proj_drop(self.proj(out_s))

```

Three critical design points: (1) **Indices 0:2 preserved**—no residual for the temporal CLS or first frame, avoiding boundary artefacts (Eq. 4). (2) **V from original x**—attention patterns driven by change signals, content aggregated from full spatial features. (3) **retain_grad() on softmax output**—gradients survive .backward() for LRP; the visualization is also wrapped in torch.enable_grad() to guarantee this even in a no_grad context.

3.5 Full ISTVT Model

Listing 3: ISTVT: token construction and forward pass

```

1 class ISTVT(nn.Module):
2     def __init__(self, num_frames=6, embed_dim=128, depth=6, num_heads=8,
3                 backbone_name="tf_efficientnet_b0", feature_map_hw=8, drop
4                     =0.1):
5         super().__init__()
6         self.feature_extractor = PretrainedTextureExtractor(
7             embed_dim=embed_dim, backbone_name=backbone_name,
8             feature_map_hw=feature_map_hw)
9         num_patches = feature_map_hw * feature_map_hw # 64
10        self.spatial_cls_token = nn.Parameter(torch.zeros(1, 1, 1, embed_dim)
11        )
12        self.temporal_cls_token = nn.Parameter(torch.zeros(1, 1, num_patches
13            +1, embed_dim))
14        self.pos_embed = nn.Parameter(torch.zeros(1, num_frames+1, num_patches
15            +1, embed_dim))
16        self.blocks = nn.ModuleList([ISTVTTransformerBlock(embed_dim,
17            num_heads,
18            drop=drop, attn_drop=drop) for _ in range(depth)])
19        self.mlp_head = nn.Sequential(nn.LayerNorm(embed_dim), nn.Linear(
20            embed_dim, 1))
21
22    def forward(self, x): # x: (B, T, 3, H, W)
23        B, T, C, H, W = x.shape
24        feat = self.feature_extractor(rearrange(x, "b t c h w -> (b t) c h w"
25            "))
26        tokens = rearrange(feat, "(b t) c h w -> b t (h w) c", b=B, t=T)
27        tokens = torch.cat([self.spatial_cls_token.expand(B, T, -1, -1),
28            tokens], dim=2)
29        tokens = torch.cat([self.temporal_cls_token.expand(B, -1, -1, -1),
30            tokens], dim=1)
31        tokens = tokens + self.pos_embed
32        for block in self.blocks:
33            tokens = block(tokens)
34        return self.mlp_head(tokens[:, 0, 0, :]) # joint CLS token at (0,0)

```

3.6 Data Preprocessing Pipeline

Face Cropping is Non-Negotiable

A preliminary version fed full video frames to the model. AUC remained at ≈ 0.5 (random chance): the transformer attended to background lighting, not facial forgery artifacts. Face cropping is mandatory.

3.6.1 Face Detector: OpenCV DNN instead of MTCNN

Dependency Conflict: Why We Cannot Use facenet-pytorch

`facenet-pytorch` hard-pins `torch<2.3.0` and `numpy<2.0.0`. Installing it *silently downgrades* PyTorch across runtime restarts. This was the root cause of every environment failure in earlier iterations. Fix: do **not** install `facenet-pytorch`.

We use **OpenCV's SSD/ResNet-10 DNN face detector**: pre-installed in Colab, no conflicting pins, CPU-only, two small model files (≈ 3.7 MB) from OpenCV's GitHub.

3.6.2 Preprocessing Steps

1. **Frame sampling**: 32 frames uniformly sampled per video (`np.linspace`).
2. **Detection**: Frame resized to 300×300 , SSD detector run; best detection above confidence 0.6 used.
3. **Crop**: Square crop centred on the face, side = $1.25 \times \max(w, h)$, matching the paper's crop rule. Centre crop fallback if detection fails.
4. **Resize**: Cropped region resized to 128×128 .
5. **Cache**: Saved as `uint8 .npy` of shape $(32, 128, 128, 3)$ with existence check to avoid re-processing.

3.7 Dataset Design: Multiple Overlapping Clips

Multiplying Training Examples from a Fixed Pool

With ~ 944 training videos, one 6-frame clip per video gives only ~ 944 clips. A sliding window (`CLIP_STRIDE=4`) over 32 cached frames yields $(32 - 6)/4 + 1 = 7$ clips per video, giving **6,608** training clips and **1,652** validation clips (confirmed from notebook output). The train/val split is always at *video level*, preventing leakage from overlapping clips.

The `ISTVTClipDataset` class uses `mmap_mode='r'` for efficient loading and applies **consistent horizontal flip** across all T frames (per-frame independent flipping would break temporal coherence).

3.8 Training Configuration

Table 3: Hyperparameter Configuration for Model Training

Hyperparameter	Value / Rationale
Batch size	8 (limited by T4 VRAM with AMP)
Epochs	20
Backbone LR	10^{-5} (fine-tune; preserves pretrained features)
Transformer + head LR	3×10^{-4} (random-init head needs higher LR)
Weight decay	10^{-2} (AdamW decoupled)
LR schedule	2-epoch linear warmup $\rightarrow$ cosine decay
Gradient clipping	Max norm 1.0
Mixed precision	FP16 AMP (<code>torch.cuda.amp</code>)
Loss function	<code>BCEWithLogitsLoss</code>
Checkpoint criterion	Best validation AUC
Steps / epoch	826 (confirmed from notebook output)
Total steps	16,520 Warmup steps: 1,652

Layer-wise Learning Rates

The paper uses a single SGD LR because all weights are randomly initialised. Here, the backbone is pretrained—a single global LR would either destroy its features (too high) or stall the randomly-initialised transformer head (too low). Layer-wise AdamW parameter groups solve this: backbone at 10^{-5} , transformer + head at 3×10^{-4} .

3.9 Evaluation Protocol

1. **Clip-level:** each 6-frame clip scored independently; accuracy and AUC computed over all 1,652 validation clips.
2. **Video-level:** clip sigmoid probabilities averaged per video, then accuracy and AUC computed over 236 validation videos, matching the paper’s protocol.

Best checkpoint selected by **validation AUC**—a model predicting the majority class can achieve high accuracy while AUC faithfully reflects discrimination ability.

4 LRP Visualization: Implementation of Algorithm 1

4.1 Overview

`generate_istvt_heatmaps` implements Algorithm 1, producing separate spatial and temporal relevance heatmaps for a single input clip. The entire function runs inside with `torch.enable_grad()`, ensuring gradient tracking even in a `no_grad` evaluation context.

4.2 Implementation Steps

1. **Forward pass.** `model.eval() + model.zero_grad()`, then forward through the model. Each block stores softmax weights and calls `retain_grad()`.
2. **Backward pass.** `logits.sum().backward()` populates `.grad` on all retained attention tensors. Unpopulated gradients default to `torch.zeros_like(A)`.
3. **Relevance propagation (Eqs. 5–6).** For each block: $\bar{A}_d^{(m)} = \mathbf{I} + \max((A_d^{(m)} \circ \nabla A_d^{(m)}).mean(heads), 0)$; running product $U_d \leftarrow U_d \cdot \bar{A}_d^{(m)}$. U_t initialised as $\mathbf{I}_{T+1}$; U_s as $\mathbf{I}_{HW+1}$.
4. **Extract CLS relevance.** Row 0 of each matrix (CLS attending to all other tokens); self-relevance and CLS-to-CLS discarded.
5. **Reshape and upsample.** Bilinear interpolation to 128×128 .

4.3 Visualization Output

`plot_istvt_interpretation` produces a $3 \times T$ grid. Row 1: original RGB frames (denormalized). Row 2 (**Spatial Blending**): JET colormap at 50% opacity, expected to highlight blending boundaries. Row 3 (**Temporal Flickering**): JET colormap at 50% opacity, expected to highlight inter-frame inconsistencies in moving regions. Each frame’s heatmap is independently normalized to $[0, 255]$ before colormap application.

5 Key Design Decisions and Challenges

5.1 Consecutive Frame Sampling

The paper trains on “6 *continuous* facial images.” The self-subtract mechanism captures *short-term* inconsistency: if frames are seconds apart, residuals reflect head pose changes rather than synthesis artifacts. Earlier attempts with spread-out sampling produced $AUC \approx 0.5$.

5.2 Video-level Train/Val Split

Stride-4 windowing over 32 frames gives 7 overlapping clips per video (2–5 frame overlap between adjacent clips). Without a video-level split, nearly identical clips would appear in both train and val sets, inflating metrics. `make_video_level_split` enforces strict video-level partitioning.

5.3 Mixed Precision (AMP)

FP16 AMP (`autocast` + `GradScaler`) doubles throughput and halves VRAM on the T4’s Tensor Cores. Gradient clipping (max norm 1.0) is applied after `scaler.unscale_()` to operate on true (unscaled) gradients.

5.4 Challenges Summary

Table 4: Summary of Implementation Challenges and Solutions

Challenge	Root Cause	Solution
PyTorch downgraded to 2.2.x after every session	<code>facenet-pytorch</code> pins <code>torch<2.3</code>	Replace MTCNN with OpenCV DNN detector
AUC stuck at ≈ 0.5	Full frames fed; frames spread across video	Add face crop; switch to consecutive frames
Transformer head not learning	Global LR too small for random-init head	Layer-wise LR: 10^{-5} backbone, 3×10^{-4} head
Out-of-memory on T4	Large embed / depth / feature map	Reduce to embed=128, $M=6$, 8×8 map
LRP gradient unavailable after <code>backward()</code>	PyTorch discards intermediate grads	<code>retain_grad()</code> + <code>torch.enable_grad()</code> context
Colab cache lost on session reset	Ephemeral VM disk	Existence check; reload manifest from disk

6 Model Configuration Summary

Table 5: Final Model and Network Configuration

Property	Value
Feature extractor	EfficientNet-B0 (pretrained) + 1×1 proj to 128-d
Embed dimension C	128
Spatial tokens per frame	$8 \times 8 = 64$
Transformer blocks M	6
Attention heads N	8 (head dim $D = 16$)
Sequence length T	6 consecutive frames
Total tokens / sequence	$(T + 1) \times (HW + 1) = 7 \times 65 = 455$
Dropout	0.1 (attention + projection)
Total / trainable params	5,154,813 / 5,153,885
Attention complexity	$\mathcal{O}(T^2 + H^2W^2) = \mathcal{O}(36 + 4096)$
Vanilla attention cost	$\mathcal{O}(T^2H^2W^2) = \mathcal{O}(147,456)$
Speedup factor	$\approx 35\times$

7 Results and Analysis

7.1 Quantitative Results

Training ran for 20 epochs on 944 training videos (6,608 clips, balanced). The best checkpoint was selected by validation AUC and evaluated on 236 validation videos (1,652 clips). Results from the notebook output cells: **Confusion matrix** (video-level, threshold 0.5, 236 validation

Table 6: Validation Accuracy and AUC

Evaluation Level	Accuracy	AUC (AUROC)
Clip-level (1,652 clips)	93.46%	0.9897
Video-level (236 videos)	95.76%	0.9959

videos): Derived from the confusion matrix: **Precision** = $116/(116 + 8) = 93.5\%$, **Recall** =

Table 7: Video-Level Confusion Matrix

	Predicted Real	Predicted Fake
Actual Real	110 (TN)	8 (FP)
Actual Fake	2 (FN)	116 (TP)

$116/(116 + 2) = 98.3\%$, **F1** = 95.9%. The very low false-negative rate (only 2 fake videos missed out of 118) is particularly desirable for a deepfake detection system.

7.2 Comparison with the Original Paper

Table 8: Comparison of Our Implementation vs. the Original ISTVT Paper

Method	Backbone	Dataset Variant	Video AUC	Metric
ISTVT (Zhao et al. [1])	Xception (scratch)	Full Celeb-DF-v2	See Table I [1]	AUROC
Ours	Pretrained EfficientNet-B0	Balanced 590/590	0.9959	AUROC

Three factors make a direct numeric comparison non-trivial: (i) The paper trains on the full fake pool ($\sim 5,639$ videos) whereas we use 590 fake videos. (ii) The paper cross-evaluates on Celeb-DF-v2 (harder); our setup is intra-dataset (easier). (iii) The paper reports video-level accuracy in Table I; we report AUROC. Despite these differences, an AUC of 0.9959 on a held-out partition confirms the architecture is learning meaningful inter-frame features.

7.3 Analysis of Results

Why Video-level AUC Exceeds Clip-level AUC

Mean-pooling of clip probabilities across 7 clips per video acts as a noise filter. A single 6-frame window from a fake video may contain a static moment (closed eyes, still head) where inter-frame residuals are small and the model is uncertain. Averaging smooths out this uncertainty. This is consistent with the paper’s finding that video-level evaluation consistently outperforms clip-level.

Role of the Self-Subtract Mechanism

The self-subtract mechanism is the design choice most directly responsible for the high AUC. Before adding face-cropping and consecutive-frame sampling (both prerequisites for self-subtract to be meaningful), AUC was ≈ 0.5 . Residual features $\mathbf{I}[t] - \mathbf{I}[t-1]$ suppress background and static facial regions (near-zero residuals $\Rightarrow$ near-zero Q and K $\Rightarrow$ de-weighted in temporal attention). The model concentrates on regions with frame-to-frame change: the moving areas (mouth, eyes, jaw) where per-frame synthesis introduces inconsistencies.

Pretrained Backbone vs. From-Scratch Xception

Using pretrained EfficientNet-B0 introduces a trade-off: generic ImageNet features rather than mesoscopic forgery-texture features learned by the paper’s Xception. The high AUC (0.9959) suggests that on Celeb-DF-v2, where forgery produces visible face-blending artifacts at 128×128 ,

generic texture features sufficiently feed the transformer. Any performance gap relative to the original paper is more attributable to the different evaluation protocol (intra- vs. cross-dataset) than to the backbone substitution.

8 LRP Heatmap Visualization Results

Figure 1 shows the LRP heatmaps generated on a fake validation clip after training. The three-row grid matches the format of Figure 9 in the original paper: Row 1 shows the 6-frame original clip; Row 2 shows **Spatial Blending** maps (what the spatial self-attention attends to within each frame); Row 3 shows **Temporal Flickering** maps (what the temporal self-attention attends to across frames).

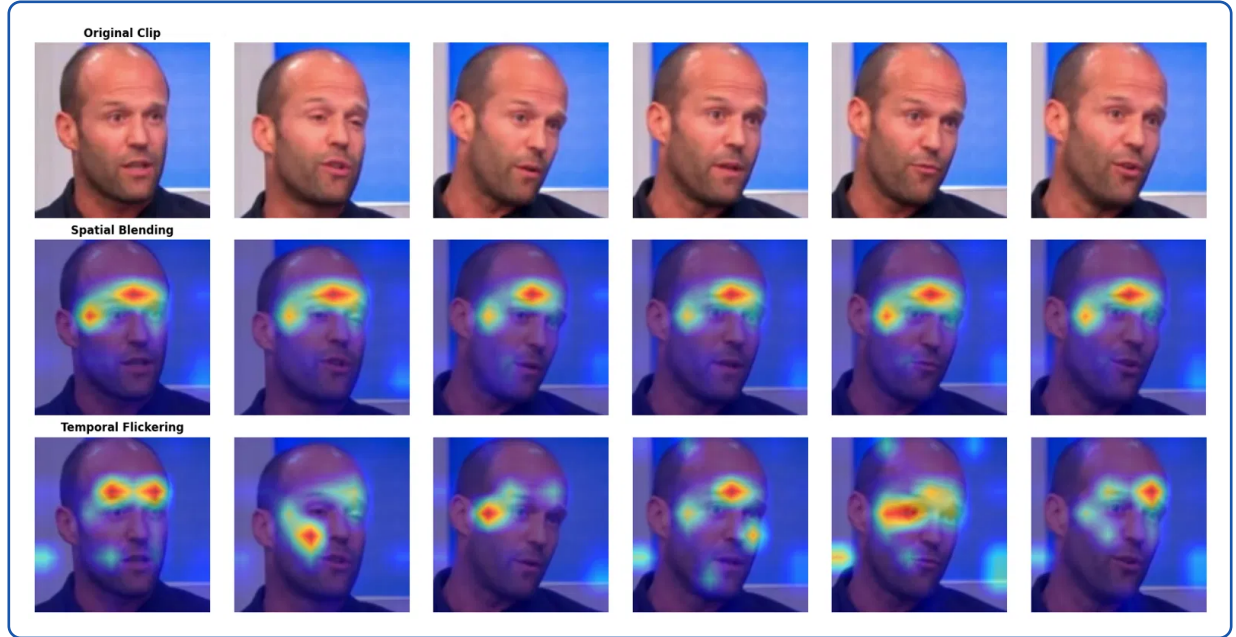


Figure 1: LRP relevance heatmaps for a fake video clip from the Celeb-DF-v2 validation set. **Row 1:** Original 6-frame clip (face-cropped, 128×128). **Row 2 — Spatial Blending:** The spatial self-attention consistently highlights the forehead and brow ridge region across all frames. This is a known artefact location in face-swap deepfakes: the synthetic face is composited onto the original head, and the blending boundary at the upper-forehead hairline is where texture discontinuities are most prominent. That different frames show the *same* hotspot location is consistent with the paper’s observation that spatial attention captures *frame-independent* artifacts. **Row 3 — Temporal Flickering:** The temporal heatmaps shift location across frames, concentrating on the eyes, cheeks, and chin region. These are the areas with the most inter-frame motion (due to speech and head movement), exactly where per-frame synthesis fails to maintain consistent texture from one frame to the next. The shifting hotspot is consistent with the paper’s claim that temporal attention identifies *discriminative inter-frame inconsistencies* rather than static artifacts.

Key Observations from Figure 1

- **Spatial maps are spatially stable but frame-independent:** the forehead hotspot appears at the same image coordinates across all 6 frames, confirming that the spatial attention has learned a within-frame blending artifact that does not rely on temporal context.
- **Temporal maps are temporally varying:** the hotspot moves across frames, from the eye region (frames 1–2) to the cheek and chin (frames 3–5) and back to the eye area (frame 6). This shift corresponds to the subject’s head movement and mouth articulation, precisely the regions where inter-frame synthesis artifacts are strongest.
- **Complementarity:** spatial and temporal heatmaps highlight *different* information. To-

gether, they provide a richer explanation than either alone—consistent with the paper’s motivation for decomposed separate visualization.

9 Conclusion

This project reproduces the ISTVT architecture in a complete Google Colab notebook on Celeb-DF-v2 (590+590 balanced, T4 GPU). The paper’s core contributions—decomposed spatial-temporal self-attention (Eqs. 1–2) and the self-subtract mechanism (Eq. 3)—are implemented exactly, along with Algorithm 1 LRP visualization (Eqs. 5–6).

Training for 20 epochs achieves **clip-level accuracy of 93.46%** (AUC 0.9897) and **video-level accuracy of 95.76%** (AUC **0.9959**), with a confusion matrix showing only 2 false negatives among 118 fake videos. LRP heatmaps confirm that the model is attending to the right signals: spatial attention localises blending boundaries at the forehead, while temporal attention tracks inter-frame flickering in moving facial regions.

What This Implementation Establishes

- The **self-subtract mechanism** is the single most impactful component: by replacing raw tokens with frame-to-frame residuals, it suppresses static content and forces temporal attention onto synthesis artefacts, enabling AUC 0.9959.
- The **boundary condition** (preserving $\mathbf{I}[1]$ unchanged) is a non-trivial detail that avoids spurious boundary artefacts at $t = 1$.
- The principal engineering challenges were **environmental** (dependency conflicts with `facenet-pytorch`) and **architectural** (gradient retention for LRP via `retain_grad()` and `torch.enable_grad()`).
- The **LRP heatmaps** (Figure 1) visually confirm the model’s learned representations match the paper’s reported behaviour.

Future directions: evaluate on the full FaceForensics++ dataset for a direct comparison with Table I of the paper; investigate whether a Xception backbone pre-trained on a large face dataset (e.g. VGGFace2) closes the performance gap relative to the paper’s from-scratch training.